WRITTEN TEST - INSTRUCTION	
Candidate Name:	
Assessor Name:	
Qualification:	**National Skill Certificate in Web Application Development with Python, Level-4**
Units of Competency	GU-11-L4-V1 Lead Small Team SU-ICT-03-L3-V1 Comply to Ethical Standards in the ICT Workplace OU-ICT-WADP-01-L4-V1 Enable Django Framework Environment OU-ICT-WADP-02-L4-V1 Develop dynamic web pages OU-ICT-WADP-03-L4-V1 Use Django model OU-ICT-WADP-04-L4-V1 Process Forms OU- ICT-WADP-05-L4-V1 Customize UI OU-ICT-WADP-06-L4-V1 Apply User Management OU-ICT-WADP-07-L4-V1 Create API using Django REST Framework OU-ICT-WADP-08-L4-V1 Create Final project
Assessment Centre:	
Date of Assessment:	
Time Of Assessment:	

Instruction's:

Read and understand the direct Wins carefully:

- this written examination is based on the performance criteria from all the units of competency in **Web Application Development with Python, Level-4**
- this assessment activity will be used to measure your underpinning knowledge
- write your answers on the paper provided
- answer all the questions as best as possible
- you have 1 hour to complete this test

Candidate's Signature		Date:
Assessor Signature		Date:

Multiple Choice Question	
1. What does the ForeignKey field represent in Django?	A. A many-to-many relationship B. A one-to-one relationship **C. A many-to-one relationship** D. A self-referential relationship
2. Which commands use to create a project in Django?	A. $ django-admin createproject project_name **B. $ django-admin startproject project_name** C. $ Django startproject project_name D. $ Django createproject project_name
3. How do you query all objects from a model?	A. Model.fetch_all() **B. Model.objects.all()** C. Model.objects.retrieve() D. Model.all()
4. How do you include another template inside a Django template?	A. {% import 'template_name.html' %} B. {% use 'template_name.html' %} **C. {% include 'template_name.html' %}** D. {% render 'template_name.html' %}
5. What is the purpose of the {% csrf_token %} tag?	**A. Adds a security token to forms** B. Renders a CSRF error message C. Verifies user authentication D. Encrypts the form data
6. How do you log out a user in Django?	A. request.logout() B. logout_user() **C. logout(request)** D. deactivate_user()
7. What is the purpose of the pip freeze command in Python?	A. It installs Python packages. **B. It lists all installed packages and their versions.** C. It upgrades the Python interpreter. D. It uninstalls Python packages.
8. What is a Django model in the context of web development?	A. A template engine **B. A database table representation** C. A CSS framework D. A JavaScript library
9. What is the default port for the Django development server?	**A. 8000** B. 8080 C. 3000 D. 5000
10. Which option is used to make a field optional in a Django model?	A. optional=True **B. null=True, blank=True** C. required=False D. nullable=True
11. Which command creates a new migration file?	A. python manage.py migrate **B. python manage.py makemigrations** C. python manage.py create_migration D. python manage.py generate_migration

12. What does @login_required in Django do?	**A. Blocks non-authenticated users from accessing a view** B. Logs in a user C. Redirects unauthenticated users to the home page D. Enables session management
13. How do you register a model in the Django admin?	A. admin.register(model_name) B. admin.site.add_model(model_name) **C. admin.site.register(model_name)** D. admin.model_register(model_name)
14. What is the purpose of serializers in DRF?	A. Validate database models **B. Convert complex data to JSON and vice versa** C. Manage migrations D. Handle URL routing
15. Which database is the default for Django projects?	A. PostgreSQL B. MySQL **C. SQLite** D. MongoDB

16. What is the difference between get() and filter() in Django QuerySet?

Ans: get( ): The get() method is used to retrieve a single object from the database. It returns only one model instance. If no record matches the query, it raises a DoesNotExist exception.

filter( ): The filter() method is used to retrieve one or more matching objects from the database. It returns a QuerySet containing all matching records. If no records are found, it returns an empty QuerySet and does not raise any exception.

```
#Get Method
profile = UserProfile.objects.get(username=username)

# Filter method
orders = Order.objects.filter(status="pending")
```

17. What are makemigrations and migrate commands in Django?

Ans: makemigrations: The makemigrations command is used to create migration files based on changes made in Django models. When a developer adds, removes, or modifies model fields, Django detects these changes and generates migration files that describe how to apply those changes to the database.

migrate: The migrate command is used to apply the migration files to the database. It executes the SQL commands required to create or modify database tables according to the migration files.

```
python manage.py makemigrations

python manage.py migrate
```

18. How do you set up a database connection in Django?

Ans: To set up a database connection in Django, we configure the DATABASES setting in settings.py. Specify the database engine, name, and other connection details. For example, to use SQLite:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

19. What are the main features of Django?

Ans: Main features include:

- Object-Relational Mapping (ORM)
- Automatic admin interface
- URL routing
- Template engine
- Form handling
- Authentication system
- Caching framework
- Internationalization support

20. What is the role of settings.py in a Django project?

Ans: The settings.py file in a Django project configures key aspects of the application, including database connections, security settings, installed apps, and middleware. For example, it specifies the database engine and name:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

21. What is Django ORM? What database operations can be performed using Django's ORM, give example?

Ans: Django ORM (Object-Relational Mapping) is a feature that allows developers to interact with the database using Python code. It translates Python objects into database tables and allows for easy querying and manipulation of data.

Creating records:

```
from myapp.models import *

new_book = Book(title='Django Basics', author='Jane Doe' )
new_book. save ()
```

Reading record:

```
all_books = Book. objects. all ()
specific_book = Book.objects.get(id=1)
```

Updating record:

```
book = Book.objects.get(id=1)
book.title = 'Updated Title'
book.save()
```

Deleting record:

```
Book.objects.get(id=1).delete()
```

22. What is the role of the 'is_authenticated' attribute in Django user authentication?

Ans: In Django, the is_authenticated attribute is a built-in attribute of the User object provided by the authentication system. This attribute is used to check whether a user is authenticated or not. When a user is successfully authenticated, the is_authenticated attribute returns True.

23. What is a Django model in the context of web development? What is the purpose of the auto_now_add option in a Django DateTimeField?

Ans: A Django model is a Python class that represents a database table in a web application. Each model defines the structure of the data, including fields and their data types, and Django automatically handles database operations such as creating tables, inserting records, updating data, and deleting records.

The auto_now_add option in a Django DateTimeField is used to automatically store the date and time when a record is created. When auto_now_add=True is set, the field is populated only once—at the time of object creation—and cannot be modified afterward.

24. What is the purpose and use of the save() function in Django?

Ans: The save() function in Django is used to store or update model data in the database. It writes a model instance to the database table associated with that model. It creates a new record if the instance is unsaved or updates an existing record if the instance is already in the database.

```
book = Book.objects.get(id=1)
book.title = 'Updated Title'
book.save()
```

25. What is the purpose of the models.py file in a Django app? How do you define a model in Django?

Ans: The models.py file in a Django app is used to define the data structure of the application. It contains Django model classes that represent database tables. Each model describes the fields, data types, and relationships of the data, and Django uses these definitions to automatically create and manage database tables through migrations.

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    created_at = models.DateTimeField(auto_now_add=True)
```

26. What is a view in Django?

In Django, a view is a function or class that handles HTTP requests and returns an HTTP response. Views are the logic behind what gets displayed on a webpage. Here's an example of a Django view that renders an HTML file:

```
from django.shortcuts import render

def home_page(request):

  return render(request, 'home.html')
```

27. How can you delete multiple records in Django ORM? Explain the use of the delete() method with an example.

Ans: In Django ORM, multiple records can be deleted using the delete() method on a QuerySet. When delete() is called on a QuerySet, Django removes all records that match the query conditions from the database in a single operation.

```
Book.objects.filter(book_type='historical').delete()
```

28. What is the purpose of serializers in DRF?

Ans: In Django Rest Framework (DRF), serializers are used to converting complex data types, such as Django models or querysets, into native Python datatypes that can be easily rendered into JSON, XML, or other content types. Additionally, serializers handle the reverse process, deserializing data from incoming requests into complex Python data types.

29. What method is used to handle submitted data by form in a Django view? How do you define a form class in django?

Ans: In Django, submitted form data is handled in a view using the HTTP POST method. When a user submits a form, the data is sent to the server as a POST request, and the Django view accesses this data using request.POST. A form class in Django is defined by creating a Python class that inherits from forms.Form or forms.ModelForm.

```
class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
```

30. What is the purpose of the 'reverse()' function in Django with respect to URLs?

Ans: The reverse() function in Django is used to generate URLs dynamically by providing the view name and optionally any parameters needed by the URL pattern. This function helps in avoiding hardcoding URLs in templates or views, making the code more maintainable and flexible. It allows developers to refer to URLs by their logical names rather than their physical paths, which simplifies URL changes and improves code readability